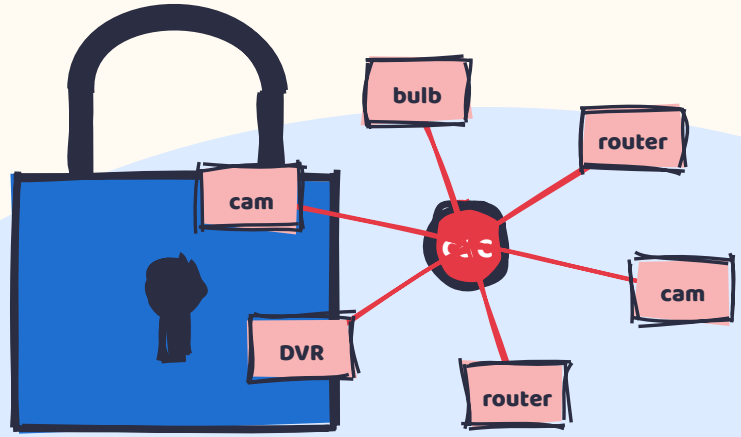
Tech Leaders · Class 13

Securing an IoT Device

Think like an attacker, build like a defender

Threat modelling | Passwords & hashing | Rate limiting | Ethics & law



Did you know?
In 2016 the Mirai botnet took over about 600,000 cameras and routers using a list of only about 60 default passwords.

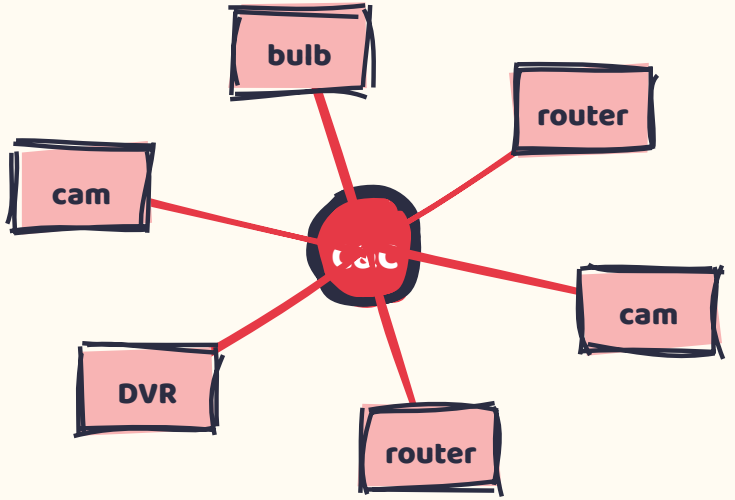
LEARN

Let's think...

Could a stranger switch off your lamp?

Last lesson's API

In TL-12 we built a lamp anyone on the Wi-Fi could control. Now imagine it is a **door lock**, a **baby camera** or a **water pump**. In 2016, hacked cameras and routers were used to knock major websites offline. How did attackers get in?



Super-genius hackers?

Secret back doors?

Default passwords!

Today: you attack and defend **your own** systems: crack weak PINs, hash passwords properly, add lockouts and audit a device.

LEARN

Learning objectives

By the end of this class I can...

1	**Threat model:** list a device's **assets, attackers and attack surface**	**Proof:** a threat table for our TL-12 lamp API
2	**Passwords:** calculate why **default and short passwords** fall in seconds	**Proof:** our keyspace table with crack times
3	**Store safely:** store passwords as **salted, slow hashes**, never plain text	**Proof:** two users with the same password get different hashes
4	**Slow attackers:** add **lockout / rate limiting** and measure its effect	**Proof:** our maths shows brute force goes from seconds to days
5	**Audit ethically:** audit a device against a **security checklist**, legally and ethically	**Proof:** an audit report with 3 fixes, for a device we own

LEARN

Objective 1 · Threat model · video

Video time!



Your Smart Home Is Stupid: IoT Security Explained

Techquickie · about 6 minutes · youtu.be/SvFyd8DpRkw

Watch for...

1. Why are cheap smart devices often **insecure**?
2. What is a **botnet** and why do attackers want your devices?
3. Name **two things** owners can do to stay safer.

Scan to watch

After the video: List every internet-connected device in your home. Which one would worry you most if a stranger controlled it?

LEARN

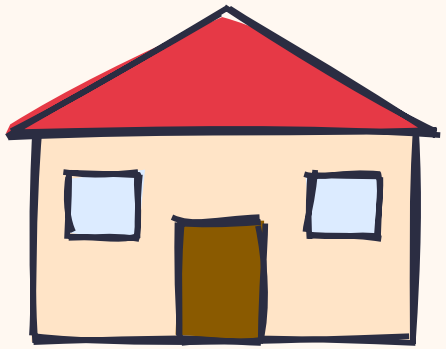
Objective 1 · Threat model

Threat modelling: 4 questions

The idea

1. **What are we protecting?** (assets: the lamp, the Wi-Fi password, sensor data)
2. **Who might attack, and why?** (a prankster, a botnet, a nosy neighbour)
3. **How could they get in?** (attack surface: open API, default password, USB port)
4. **What will we do about it?** (fixes, ranked by risk = likelihood × impact)

Example: protecting your house



You don't buy a vault door and leave the window open. You list the ways in (doors, windows, roof), who might try, and fix the easiest and most dangerous first.

LEARN

Objective 1 · Threat model · team activity

Threat-model our TL-12 lamp API

Threat	Likelihood	Impact	Fix
Anyone on Wi-Fi sends POST /api/lamp	high	high	
Wi-Fi password written in the code on GitHub			
Someone floods the API with requests			
Someone reads traffic on the network			
Someone unplugs it and reflashes it via USB			

Score it

Rate likelihood and impact as low / medium / high. Fix the **high × high** ones first.

Points

+1 for each new threat no other team found!

LEARN

Real world

When IoT security failed



Mirai (2016)
Default passwords → 600k devices → huge outage



Jeep (2015)
Researchers remotely controlled a car's brakes and steering



Pacemakers (2017)
465,000 recalled for a security firmware update



Casino fish tank
Hackers entered a casino network through a smart thermometer

? What do all four have in common?

Lesson
Every device on a network is a possible door. The weakest one decides how safe everyone is.

LEARN

Objective 2 · Passwords

Keyspace: how many guesses?

The idea

Combinations = characters ^ length

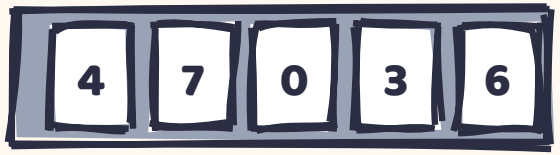
4-digit PIN: 10^4 = **10,000**

8 lowercase letters: 26^8 ≈ **209 billion**

12 mixed (a-z, A-Z, 0-9): 62^{12} ≈ **3×10^{21}**

A laptop can try millions of guesses per second against a stolen file. Length beats complexity: adding characters multiplies the work enormously.

Example: a bicycle lock



A 3-dial lock has 1,000 combinations: a bored thief tries them all in 20 minutes. A 5-dial lock has 100,000: that takes days. Each extra dial multiplies the work by 10.

LEARN

Objective 3 · Store safely

Never store passwords. Store hashes.

The idea

A **hash** is a one-way fingerprint: easy to compute, practically impossible to reverse. To log in, hash what the user typed and compare.

Add a random **salt** per user so identical passwords get different hashes, and use a deliberately **slow** hash (PBKDF2, bcrypt, Argon2) so each guess costs the attacker time.

Slow is good
Plain SHA-256 is fast, which is BAD for passwords. Fast hashes let attackers try billions per second.

Example: a fingerprint



Police can match a fingerprint to a person, but they can't rebuild the person from the print. A hash lets the device check a password without ever keeping the password itself.

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Hack only what you own
2. Salted, slow hashing in Python
3. Crack a PIN, then do the maths
4. Lockout turns seconds into days
5. IoT security checklist
6. Write a responsible audit report
7. Results log

Before we start

Hack only what you own

The rule

Today we only test **our own** code, **our own** devices, on the **lab network**, with the teacher's permission.

Accessing someone else's device, account or network without permission is a **crime** in Pakistan under the Prevention of Electronic Crimes Act (PECA) 2016, and in almost every country.



Ethical hackers

Companies pay 'penetration testers' and bug bounty hunters to find weaknesses, with written permission.

MAKE

Objective 3 · Store safely · code it

Salted, slow hashing in Python

```python
import hashlib, os, hmac

def hash_password(pw, salt=None):
    salt = salt or os.urandom(16)                # random per user
    h = hashlib.pbkdf2_hmac("sha256", pw.encode(), salt, 200_000)
    return salt, h

def check_password(pw, salt, stored):
    return hmac.compare_digest(hash_password(pw, salt)[1], stored)

users = {}
for name in ["ali", "sara"]:
    users[name] = hash_password("Robot#2026")      # same password!

for name, (salt, h) in users.items():
    print(name, salt.hex()[:8], h.hex()[:16])      # different hashes

salt, h = users["ali"]
print(check_password("robot", salt, h))           # False
print(check_password("Robot#2026", salt, h))      # True
```

200,000 rounds

Makes each guess take a fraction of a second for us, but turns billions of guesses into years for an attacker.

compare_digest

Compares in constant time, so attackers can't learn anything from how long the check takes.

Check

Same password, different salts, different hashes. A leaked file doesn't show who shares a password.

MAKE

Objective 2 · Passwords · code it

Crack a PIN, then do the maths

```python
import itertools, time

secret = "4821"                          # our own test PIN
start = time.time()
for guess in itertools.product("0123456789", repeat=4):
    if "".join(guess) == secret:
        break
print("Cracked in", round(time.time() - start, 4), "seconds")

rate = 1_000_000                          # guesses per second (offline)
for chars, n, name in [(10, 4, "4-digit PIN"), (26, 8, "8 lowercase"),
                       (62, 8, "8 mixed"), (62, 12, "12 mixed"),
                       (7776, 5, "5 random words")]:
    combos = chars ** n
    days = combos / rate / 86400
    print(f"{name:15} {combos:.2e} combos  ~{days:.2e} days")
```

Seconds!

A 4-digit PIN with no lockout falls in well under a second on a laptop.

Passphrases

5 random words from a 7,776-word list beat 12 random characters, and are easier to remember.

Default = zero

If the password is 'admin', the keyspace is effectively 1. The attacker just reads the manual.

MAKE

Objective 4 · Slow attackers · code it

Lockout turns seconds into days

```
import time
SECRET, MAX_TRIES, LOCK_SECONDS = "4821", 3, 60
tries, locked_until = 0, 0

def login(pin):
    global tries, locked_until
    if time.time() < locked_until:
        return "LOCKED"
    if pin == SECRET:
        tries = 0
        return "OK"
    tries += 1
    if tries >= MAX_TRIES:
        locked_until = time.time() + LOCK_SECONDS
        tries = 0
    return "WRONG"

# how long would brute force take now?
hours = 10_000 / MAX_TRIES * LOCK_SECONDS / 3600
print("Worst case:", round(hours), "hours")       # about 56 hours
```

The maths

3 guesses per minute → 10,000 PINs take about **56 hours** instead of under a second.

Side effect

An attacker could lock out the REAL owner on purpose. Real systems grow the delay step by step and alert the owner.

On the ESP32

The same idea fits in the TL-12 API: count failed keys and reply **429 Too Many Requests**.

Objective 5 · Audit ethically · audit

IoT security checklist

Check	✓ / ✗	Fix
No default or hard-coded passwords		
Secrets not in code on GitHub (use a secrets.h file in .gitignore)		
Only needed services and ports are open		
Traffic encrypted (HTTPS / TLS) where possible		
Login attempts limited		
Firmware can be updated, and is		
Device fails safe (e.g. lock stays locked on power loss)		

Based on
The OWASP IoT Top 10, a list security professionals use worldwide.

MAKE

Objective 5 · Audit ethically · report

Write a responsible audit report

Report format

Device: our TL-12 lamp API (owned by our team)
Finding: anyone on the Wi-Fi can switch the lamp
Risk: high likelihood, low impact (it is a lamp), HIGH if it were a lock
Fix: API key + lockout + move secrets out of the code
Re-test: request without key → 401 ✓

Responsible disclosure

Found a bug in a real product? Report it privately to the maker, give them time to fix it, and never use it. Many companies have a security contact page for this.

MAKE

Test like an engineer

Results log

Measure	Result	Notes
Threats found (our team / class)		
Time to crack 4-digit PIN		s
Days for 12 mixed characters		
Same password → different hashes?		
Brute force time with lockout		hours
Checklist: ✓ count / 7		

Debug checklist

pbkdf2 slow? That is the point! Lower rounds only while testing.
f-string error? You need Python 3.6 or newer.

Security is never 'done'. Each fix moves the attacker to the next weakest point.

MAKE

Level up!

Security challenges

Choose your challenge



Harden TL-12: add the API key, lockout and a secrets.h file to your ESP32 API.



Hash on the ESP32: store a salted hash of the admin password instead of the password.



Log attacks: record every failed attempt with time and IP. Spot patterns.



Network split: research why experts put IoT devices on a separate 'guest' Wi-Fi.

Think like an engineer

Why do attackers prefer to attack thousands of cheap devices rather than one well-protected server?

Boss level

Swap hardened APIs with another team and try (only) the attacks on our checklist. Report responsibly.

Debate

Who is responsible for insecure devices?

The maker

Shipped it with 'admin/admin' and no updates.

The owner

Never changed the password or updated it.

The government

Could require security standards by law (the UK and EU now do).

? Rank the three from most to least responsible. Defend your ranking in 1 minute. Should cheap devices be banned from sale if they cannot be updated?

Objective check

Can I really do it? Prove it.

01	Show your threat table. Which threat did you rank highest, and why?	○ ○ ○
02	How long would a 4-digit PIN last without lockout? And 12 mixed characters?	○ ○ ○
03	Show two users with the same password but different hashes. Why does that matter?	○ ○ ○
04	Explain how lockout changes brute-force time. What is its downside?	○ ○ ○
05	Read one finding from your audit report, including the fix and re-test.	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Security review board: teams present their highest risk and fix; the class votes on the best fix.

Show what you know

What did we learn?

- [x] Threat model: assets, attackers, attack surface, fixes
- [x] Long passwords and passphrases beat short complex ones
- [x] Store salted, slow hashes, never passwords
- [x] Lockouts and rate limits make brute force impractical
- [x] Test only what you own; report problems responsibly

Exit ticket: a device stores passwords in plain text. What could go wrong if its memory is read, and how do you fix it?

At home

Check one device at home: has its default password been changed? (Ask the owner first!)



Next class: Autonomous Fire-Fighting Robot!